

Self-adaptive Fuzzy-Timed Systems *

César Andrés, Luis Llana, Manuel Núñez

Departamento de Sistemas Informáticos y Computación

Universidad Complutense de Madrid, Spain

c.andres@fdi.ucm.es, llana@sip.ucm.es, mn@sip.ucm.es

Abstract

We consider the formal representation and analysis of systems with fuzzy-time information. First, we present a formalism to represent specifications. This model exploits the concepts of fuzzy set theory and uses a mathematical framework to get a more flexible approach.

As it is usually assumed in industrial case studies, we consider that the original requirements of the specification may change. The implementation is built with respect to these changes but the specification is not upgraded. Thus, it may be outdated. In order to continue using the formal framework, the specification must be adapted with respect to these new requirements. We consider that this update process should be as non-intrusive as possible, that is, without using the source-code of the implementation. We present a novel methodology for self-evolving fuzzy-time systems, without interacting with the source code.

Keywords: Timed Automata, Fuzzy Set Theory, Evolutive Systems.

1 Introduction

With the growing significance of computer systems, techniques that assist in the production of reliable software are becoming increasingly more important. The complexity of many computer systems requires the application of a battery of such techniques. Two of the most promising approaches are *formal methods* and *testing*. The links between testing and formal methods go well beyond generating tests from a *formal specification*. The presence of a formal specification of a system allows us to apply mathematical techniques in order

to semi-automatically perform some tasks involved in *testing* [10, 7, 6, 18] through the use of tools [20]. The specification is represented by a formal model that expresses the system requirements. A first family of requirement focuses on *what* the system does. A second family focuses on *how* the system does what it does. Even though formal testing originally concentrated on the first kind of requirements, during the last years formal testing also dealt with the second kind. In this paper we focus on systems that contain temporal restrictions, where much work has been recently developed (for example [11, 2, 19, 4, 14, 13, 5, 21, 8]).

The presence of a formal specification makes possible for the tester to understand better what passing a test means for a system. Moreover, with a formal specification we can validate if a battery of tests that we had in a previous version of the system is still sound to analyze the current version of the implementation. Nevertheless, it is very frequent that we may have an outdated specification, becoming useless. When the system is being implemented, sometimes the implementers perform small changes that are not included in the specification. In particular, these changes appear in two typical situations. The first one are introduced by implementing changes requested by the customer. That is, the client during the implementation of the system, requires an immediate change in the behavior of the system. The second one happens when a version of a product is finished and the source code is being upgraded by people outside the initial project.

This paper focuses on studying how to adapt a specification to new requirements by evolving it. On the one hand, we present a formal model to represent the different requirements of a system, that is, the specification of the system, that can present temporal behavior defined in *fuzzy* terms. On the other hand, we present an automatic mechanism to adapt and evolve specifications in changing environments.

The mathematical model used in this paper allows us to represent timed information (i.e, the time that

*Research partially supported by the Spanish MCYT project TESIS project (TIN2009-14312-C02-01) and the UCM-BSCH programme to fund research groups (GR58/10 group number 910606).

the system needs to perform certain actions) and the interaction of the system with the environment. The paradigms to represent time information in a model usually are divided into the ones that use a discrete domain (for example [17, 16]) and the one using a dense domain (for example [22, 1, 3]). The authors in favor of a dense time domain argue that its expressive power is greater than the one of a discrete time domain. Those in favor of a discrete time domain argue that the approach is more realistic since, for instance, you cannot measure $\sqrt{2}$ seconds. However, neither the discrete nor the dense models of time are capable to properly model the concept of time that humans beings have, which it is inherently vague. In this paper, we use of a structure that wraps the concept itself assuming uncertainty and vagueness. In this context *vagueness* is not necessarily a criticism, but just a fact.

Our perception of reality is not perfect, although statements can be still *true* or *false*; our *environment* can make us doubt about a truth assessment. All measurement devices have an intrinsic error: if a thermometer indicates that the temperature is 35.4 degrees, we know that the actual temperature is around that measurement. Something similar happens with time. We can claim that *it takes us an hour to go to work*. If the trip lasts 57 minutes in a particular day we know that the trip has lasted as usual, otherwise if it takes us 81 minutes we know that the trip has not lasted as usual. We can also be more accurate and give a degree of *confidence* of the measurement, which is a number in the interval $[0, 1]$ with 1 being the maximum degree of confidence and 0 the minimum. In this way, we can assess that 57 minutes is equal to 1 hour with a confidence degree close to 1, while 81 minutes has a confidence degree close to 0. *Fuzzy logic* allows us to represent this notion of time.

Let us note that fuzzy logic is not fuzzy [23]. Basically, fuzzy logic is a precise logic of imprecision and approximate reasoning. Paradoxically, one of the principal contributions of fuzzy logic is its high power of precision of what is imprecise. Several mathematics notions can be translated into the corresponding fuzzy domain in an uniform way. For instance, there are studies on well-known theories developing new logical systems for dealing with vagueness, imprecision and uncertainly [12]. In addition to the contributions in theoretical field, there are practical applications of fuzzy logic. There exist tools, such as [15], for knowledge representation and reasoning that implement fuzzy logic theory, decision-making, etc. It is important to mention that even though standard approaches to decision-making problems assume that all the information is represented in the same format, in real practice this is

hardly possible. Thus, there are fuzzy-studies [9] to integrate different preference representation on data mining, information retrieval, recommendation systems, etc.

In this paper we present a framework that make use of current fuzzy logic theory. The advantages of our framework with respect to others is that we can represent the behavior of fuzzy-timed systems. We introduce conformance relations in order to compare different models. In addition, we present an algorithm to self-refresh the information presented in a model up to a set of behaviors collected from a running system.

The rest of the paper is organized as follows. In Section 2 we present the notation we use along the paper about fuzzy theory. In Section 3 we introduce our fuzzy-timed automata model that can be used to represent specification of systems. Section 4 shows how our models are confronted with the observed execution traces and it is updated. Finally, Section 5 presents the conclusions of the paper and some lines for future work.

2 Preliminaries

We do not assume that the reader is familiar with the concepts of fuzzy logic. Therefore, we present some of its basic concepts: *fuzzy relations* and *triangular norms*.

2.1 Fuzzy relations

Fuzzy relations play an important role in fuzzy modeling, fuzzy diagnosis, and fuzzy control. In ordinary logic, a *set* or a *relation* is determined by its characteristic function, that is, a function that returns **true** if the element is in the set (or if some elements are related) and **false** otherwise. Working with a fuzzy framework, we do not have that clear distinction between truth and falsehood. Instead of this, we have a complete range of values in the interval $[0, 1]$. The larger this value, the more confidence we have in the assessment.

In this paper we only consider relations over the set of real numbers $\mathbb{R}$. Thus, a fuzzy relation is a mapping from the Cartesian product $\mathbb{R}^n$ into the interval $[0, 1]$.

Definition 1 A *fuzzy relation* $\check{A} : \mathbb{R}^n \rightarrow [0, 1]$ is a function that provides the confidence of an element in $\mathbb{R}^n$.

Let $x \in \mathbb{R}^n$. We say that x is *not included* in $\check{A}$ if $\check{A}(x) = 0$. Consequently, we say that x is *fully included* in $\check{A}$ if $\check{A}(x) = 1$. The *kernel* of $\check{A}$ is the set of elements that are fully included in $\check{A}$. $\square$

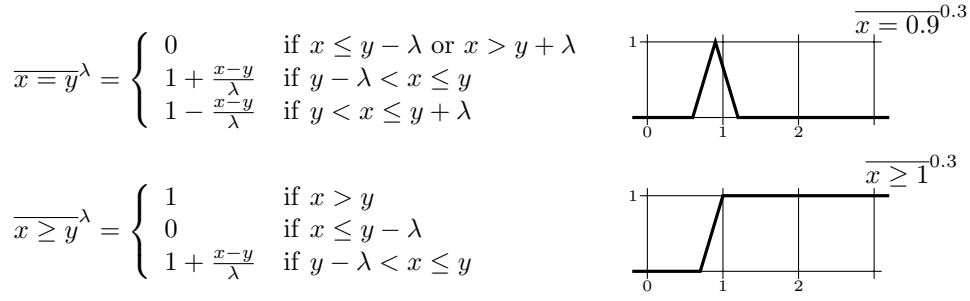


Figure 1. Fuzzy Relations.

The notion of α -cut is an important concept in fuzzy logic. Intuitively it establishes a credibility threshold. If we have a fuzzy relation $\check{A}$, we accept that the relation is **true** for x if $\check{A}(x)$ is above the threshold α .

Definition 2 Let $\check{A}$ be a fuzzy relation and $\alpha \in [0, 1]$ be a threshold value. We define the α -cut of $\check{A}$, denoted by $\text{cut}_\alpha(\check{A})$, as:

$$\text{cut}_\alpha(\check{A}) = \{x \in \mathbb{R}^n \mid \check{A}(x) \geq \alpha\}$$

□

Example 1 Let us consider $\lambda \geq 0$ be a non negative real number. In Figure 1 we present some fuzzy relations. These relations are explained by a simple study from daily life using discrete fuzzy sets to describe the relationship between the color of a fruit and the grade of maturity. When one represents the colors on a color spectrum scale, the colors would be described by their spectral distribution curves that can be interpreted as membership functions and then a particular color is a fuzzy term. For instance the color red could be represented by the first relation in Figure 1. This functions means that if we observe a value of 0.9 then it is red with confidence 1, but if we observe a value of 0.85 then we can assume to be the correct color with a degree of confidence. The second function presented in Figure 1 treats also the grades of maturity as fuzzy terms. This function represents that a fruit starts being mature on time bigger than 0.7 and it is completely mature on time 1, and all the values between 0.7 and 1 have associated a degree of confidence on this decision. □

2.2 Triangular Norms

A *triangular norm* (abbreviated *t-norm*) is a binary operation used in fuzzy logic to generalize the

conjunction in propositional logic. In order to generalize conjunction, we have to analyze its basic properties: commutativity $p \wedge q = q \wedge p$, associativity $(p \wedge q) \wedge r = p \wedge (q \wedge r)$, identity **true** $\wedge p = p$, and nilpotency **false** $\wedge p = \text{false}$.

Therefore, we require a *t-norm* to satisfy similar properties. We also require an extra property: monotonicity. Intuitively, the resulting truth value does not decrease if the truth values of the arguments increase.

Definition 3 A *t-norm* is a function $T : [0, 1] \times [0, 1] \rightarrow [0, 1]$ which satisfies the following properties:

Commutativity. $T(x, y) = T(y, x)$.

Monotonicity. $T(x, y) \leq T(z, u)$ if we have both $x \leq z$ and $y \leq u$.

Associativity. $T(x, T(y, z)) = T(T(x, y), z)$.

Neutrality of 1. $T(x, 1) = x$.

Nilpotence of 0. $T(x, 0) = 0$.

□

Let us remark that the property $T(x, 0) = 0$ can be easily deduced from the others. Since *t-norms* have the associativity property, we can generalize them to lists of numbers. Therefore, $T(x_1, x_2, \dots, x_{n-1}, x_n)$ is a shorthand for $T(x_1, T(x_2, \dots, T(x_{n-1}, x_n))) \dots$.

Example 2 Some important *t-norms* are:

Lukasiewicz *t-norm*: $T(x, y) = \max(0, x + y - 1)$.

We represent this *t-norm* with the symbol $\wedge$.

Gödel *t-norm*: $T(x, y) = \min(x, y)$. We represent this *t-norm* with the symbol $\bar{\wedge}$.

Product t -norm: $T(x, y) = x \cdot y$ (real number multiplication). We represent this t -norm with the symbol $\star$.

We assume that any used t -norm has a symbol, and we use the symbol $\triangle$ to represent a generic t -norm. We denote by $\llbracket \triangle \rrbracket$ the function associated with the symbol $\triangle$. For instance, $\llbracket \wedge \rrbracket$ is the min function. $\square$

3 Fuzzy-Timed Automata

In this Section we introduce the basic notions of fuzzy-timed automata for modeling our systems. But first, in order to make the paper self-contained, we recall some common definitions of timed automata. Later, we use and adapt these concepts to define fuzzy-timed automata. We start by defining what are the main concepts concerning time underlying: *clocks* and *constrains* over these clocks. Clocks are variables to denote the pass of time. Constrains over these clocks are boolean requirements over their valuations.

Definition 4 We assume that we have a finite alphabet of *actions*. The set of actions is denoted by **Acts**.

A *clock* is a real valued variable. We assume that we have a finite set of clocks denoted by **Clocks**.

A *clock valuation* $u : \text{Clocks} \rightarrow \mathbb{R}^+$ assigns non-negative real values to the clocks. We denote by 0 the clock valuation where all clocks are set to 0.

We use the following notation for valuations.

- Let u be a valuation and let $d \in \mathbb{R}^+$. The valuation $u + d$ denotes the valuation $(u + d)(x) = u(x) + d$.
- Let u be a valuation and r a set of clocks. We denote the reset of all clocks of r in u , written as $u[r]$, as a valuation that maps all clocks in r to 0 and leaves invariant the rest of clocks.

A *clock constraint* is a formula consisting of conjunctions of atomic relations of the form $x \bowtie n$ or $x - y \bowtie n$ where $n \in \mathbb{N}$, $x, y \in \text{Clocks}$ and $\bowtie \in \{\leq, <, =, >, \geq\}$. We denote the set of clock constraints by $\mathcal{C}$.

Let u be a valuation and $C \in \mathcal{C}$, we write $u \models C$ when the valuation u makes the clock constraint C **true**. $\square$

In the rest of the paper actions are ranged over $a, b, c, \dots$, clocks are ranged over $x, y, z, \dots$, and the valuations of the clocks are ranged over $u, v, \dots$.

With previous notation next we introduce the notion of *timed automaton*.

Definition 5 A *timed automaton* is a tuple $(L, l_0, \text{Acts}, E, I)$ where: L is a finite set of locations, $l_0 \in L$ is the initial location. **Acts** is the set of

actions of this automata, $E \subseteq L \times \text{Acts} \times \mathcal{C} \times 2^{\text{Clocks}} \times L$ is the set of edges and $I : L \rightarrow \mathcal{C}$ is a function that assigns invariants to locations. We write $l \xrightarrow{a, C, r} l'$ whenever $(l, a, C, r, l') \in E$.

As usual, *invariants* of locations consist of conjunctions of atoms of the form $x \leq n$, where $n \in \mathbb{N}$.

The *operational semantics* of a timed automaton is a timed labeled transition system whose states are pairs (l, u) , where l is a location, u is a valuation of clocks, and its transitions are:

- $(l, u) \xrightarrow{d} (l, u + d)$ if $d \in \mathbb{R}^+$ and $u + d \models I(l)$.
- $(l, u) \xrightarrow{a} (l', u[r])$ if $l \xrightarrow{a, C, r} l'$ and $u \models C$ and $u[r] \models I(l')$.

Let $A = (L, l_0, \text{Acts}, E, I)$ be a timed automaton. We say that the sequence $t = (d_1, a_1)(d_2, a_2) \dots (d_n, a_n) \in (\mathbb{R}^+ \times \text{Acts})^n$ is a *trace* of A if the following transitions exist: $(l_0, 0) \xrightarrow{d_1} (l'_0, u'_0) \xrightarrow{a_1} (l_1, u_1) \xrightarrow{d_2} (l'_1, u'_1) \xrightarrow{a_2} (l_2, u_2) \dots \xrightarrow{d_n} (l'_{n-1}, u'_{n-1}) \xrightarrow{a_n} (l_n, u_n)$. $\square$

Let us remark that in the ordinary timed automata theory [1], time constraints consist of conjunctions of inequalities. Instead of the ordinary inequalities in Definition 5, we use their fuzzy counterparts appearing in Example 1. We could have more freedom in allowing general convex fuzzy sets, but we have preferred to keep our constraints close to the original ones so we can use the theory developed for timed automata.

As we have already said, the role of conjunction in fuzzy theory is played by t -norms. There is not a *canonical* t -norm and in Example 2 we have shown three of the more used t -norms.

Time constraints can be divided in two groups: general fuzzy constraints, that appear as constraints attached to the actions, and the set of restricted constraints attached to location invariants, where only inequalities of the form $\overline{x \leq n}^\lambda$ are allowed.

Definition 6

1. A *fuzzy constraint* is a formula built from the following B.N.F.:

$$C ::= \text{true} \mid C_1 \triangle C_2 \mid \overline{x \bowtie n}^\lambda \mid \overline{x - y \bowtie n}^\lambda$$

where $\triangle$ is a t -norm, $\bowtie \in \{\leq, =, \geq\}$, $x, y \in \text{Clocks}$, $\lambda \in \mathbb{R}^+$, and $n \in \mathbb{N}$. We denote the set of fuzzy constraints by $\mathcal{FC}$.

2. A *fuzzy restricted constraint* is the subset of fuzzy constraints defined by the following B.N.F.:

$$C ::= \mathbf{true} \mid C_1 \triangle C_2 \mid \overline{x \leq n}^\lambda$$

where $\triangle$ is a t -norm, $x \in \mathbf{Clocks}$, $\lambda \in \mathbb{R}^+$, and $n \in \mathbb{N}$. We denote the set of restricted fuzzy constraints by $\mathcal{RFC}$.

□

In timed automata theory, constraints have two different uses: to decide if the automata can stay in a location and to decide if a transition can be executed. All this is done by checking if a valuation satisfies the corresponding constraint. In fuzzy theory the notion of satisfaction is not crisp, since we do not have a boolean answer, but a range in the interval $[0, 1]$. Therefore, we do not have the notion of a constraint evaluating to **true** or **false** but a *satisfaction grade* of a constraint.

Definition 7 Let u be a clock valuation and C be a fuzzy constraint. We inductively define the *satisfaction grade of C in u* , denoted by $\mu_C(u)$, as:

$$\mu_C(u) = \begin{cases} 1 & \text{if } C = \mathbf{true} \\ \overline{u(x) \bowtie n}^\lambda & \text{if } C = \overline{x \bowtie n}^\lambda \\ \overline{u(x) - u(y) \leq n}^\lambda & \text{if } C = \overline{x - y \leq n}^\lambda \\ \llbracket \triangle \rrbracket(\mu_{C_1}(u), \mu_{C_2}(u)) & \text{if } C = C_1 \triangle C_2 \end{cases}$$

where $\bowtie \in \{\leq, =, \geq\}$.

□

Let us note that in the previous definition it is easy to check that $\mu_C(u) \in [0, 1]$. Once the time constraints are defined, the definition of fuzzy-timed automata is quite straightforward. It consists in replacing ordinary time constraints by fuzzy time constraints.

Definition 8 A *fuzzy-timed automaton* is a tuple $(L, l_0, \mathbf{Acts}, E, I)$ where: L is a finite set of locations. $l_0 \in S$ is the initial location, $\mathbf{Acts}$ is the set of actions of this automaton, $E \subseteq L \times \mathbf{Acts} \times \mathcal{FC} \times 2^{\mathbf{Clocks}} \times L$ is the set of edges and $I : L \rightarrow \mathcal{RFC}$ is a function that assigns invariants to locations. We write $l \xrightarrow{a, C, r} l'$ whenever $(l, a, C, r, l') \in E$.

Let us note that clock constraints appeared in the transitions have the general form indicated in Definition 6-1, while the location invariants have the restricted form indicated in Definition 6-2.

□

Next we are going to define the operational semantics of fuzzy-timed automata. We need it to obtain the

fuzzy traces that are used for the conformance relations. This operational semantics is given in terms of transitions, which are enabled when time constraints hold. Since we do not have crisp time constraints, the transitions must be decorated with a real number $\alpha \in [0, 1]$. This number indicates its certainty.

In order to define the operational semantics we need a t -norm. Let us explain the reason. In the definition of the operational semantics of an ordinary timed automaton, action transitions require the condition “ $u \models C$ and $u[r] \models I(l')$ ”. This conjunction must be transformed into its fuzzy version: a t -norm.

Definition 9 Let $fA = (L, l_0, \mathbf{Acts}, E, I)$ be a fuzzy-timed automaton and $\triangle$ be a t -norm. The $\triangle$ -operational semantics of fA is the labeled transition system whose states are $(l, u) \in L \times \mathbf{Clocks}$, the initial state is $(l_0, 0)$, and the transitions are:

1. $(l, u) \xrightarrow{d} \alpha (l, u + d)$ whenever $\mu_{I(l)}(u + d) = \alpha$
2. $(l, u) \xrightarrow{a} \alpha (l', u[r])$ whenever $l \xrightarrow{a, C, r} l'$, $\alpha_1 = \mu_{I(l)}(u[r])$, $\alpha_2 = \mu_C(u)$ and $\alpha = \llbracket \triangle \rrbracket(\alpha_1, \alpha_2)$.

□

This operational semantics is not a pure timed transition system because the transitions are decorated with a real number $\alpha \in [0, 1]$ indicating the certainty to be executed. It is desirable that our operational semantics induce graphs that have the main properties of timed transition systems: time determinism and time additivity. In order to prove the following results, it is enough to take into account that time transitions do not change location and the clock valuation is increased with the passing of time.

Proposition 1 Let $fA = (L, l_0, \mathbf{Acts}, E, I)$ be a fuzzy-timed automaton and $\triangle$ be a t -norm. The $\triangle$ -operational semantics of fA holds the following properties:

Determinism. If $(l, u) \xrightarrow{d} \alpha_1 (l_1, u_1)$ and $(l, u) \xrightarrow{d} \alpha_2 (l_2, u_2)$, then $\alpha_1 = \alpha_2$, $l_1 = l_2$ and $u_1 = u_2$.

Additivity. If $(l, u) \xrightarrow{d_1} \alpha_1 (l_1, u_1) \xrightarrow{d_2} \alpha_2 (l_2, u_2)$ then $(l, u) \xrightarrow{d_1 + d_2} \alpha_2 (l_2, u_2)$

□

As a further remark, let us note the α_2 index of the transition $(l, u) \xrightarrow{d_1 + d_2} \alpha_2 (l_2, u_2)$ in the time additivity property. In this case, we do not have to combine it with the parameter α_1 that appears in the preceding transition because the constraints considered in the

locations have the form $\overline{x \leq n}^\lambda$. Therefore, $\alpha_1 \geq \alpha_2$. Intuitively, if the automaton stays in a location, then the likelihood of remaining in it can only decrease.

This operational semantics allows to define the traces of a fuzzy-timed automaton. These traces are sequences of pairs, where the first element of the pair expresses how long the automaton has stayed in a location and the second one has the action that has produced a location change. All these transitions are decorated with the certainty of being performed. So, whenever the automaton stays in a location, we have an $\alpha \in [0, 1]$ indicating the certainty of the automaton to stay in the location; and when an action transition is performed, we need a $\beta \in [0, 1]$ indicating its certainty.

Definition 10 Let $fA = (L, l_0, \text{Acts}, E, I)$ be a fuzzy-timed automaton and let Δ be a t -norm. We say that $t = (d_1, \alpha_1, a_1, \beta_1)(d_2, \alpha_2, a_2, \beta_2) \dots (d_n, \alpha_n, a_n, \beta_n) \in (\mathbb{R}^+ \times [0, 1] \times \text{Acts} \times [0, 1])^*$ is a Δ -fuzzy trace of fA , we denote by $\text{ftr}_\Delta(fA)$ the set of Δ fuzzy traces of fA , if there is a sequence of transitions $(l_0, 0) \xrightarrow{d_1} \alpha_1 \bullet \xrightarrow{a_1} \beta_1 (l_1, u_1) \xrightarrow{d_2} \alpha_2 \bullet \xrightarrow{a_2} \beta_2 (l_2, u_2) \dots \xrightarrow{d_n} \alpha_n \bullet \xrightarrow{a_n} \beta_n (l_n, u_n)$ in the Δ -operational semantics of fA .

In the previous sequence of transitions we have not specified the bullet states $\bullet$ to simplify the reading since they are always determined by the previous state:

$$(l_i, u_i) \xrightarrow{d_{i+1}} \alpha_{i+1} (l_i, u_i + d_{i+1}). \quad \square$$

4 Updating Fuzzy-Timed Automata

In this section we present how we perform the update of a specification. We consider that we have an initial model. We use traces collected from an implementation in order to update this specification model. The upgrade model focus on updating both behavioral and time information. If the trace represents a new behavior, then a new functional feature is introduced in the specification. On the contrary, if the trace was previously considered in the model, then only time information is adapted. We have to compare these traces with the ones that can be performed if we consider the specification. We will give two alternative *conformance relations* to manage fuzzy relations because these relations depend on two t -norms. After that, we give the algorithm that receives a model (the specification), a trace, and two t -norms and returns the modified model.

Let us note that in order to include fuzzy values, we have to adopt some assumptions. As said before, we consider that implementations are real-time systems. When we check these systems we obtain real time measurements. We need to check

if those measurements meet the fuzzy constraints given by the fuzzy specification. These traces are of the form $(d_1, a_1)(d_2, a_2) \dots (d_n, a_n)$ where the token d_i, a_i represents that the action a_i was performed on time d_i . We have to compare them with traces of the fuzzy automaton that have the form $(d_1, \alpha_1, a_1, \beta_1)(d_2, \alpha_2, a_2, \beta_2) \dots (d_n, \alpha_n, a_n, \beta_n)$. In order to have a true/false assessment, first we have to combine the certainty values $(\alpha_1, \beta_1, \alpha_2, \beta_2, \dots, \alpha_n, \beta_n)$ with a t -norm and to compare the result to a given threshold $\alpha \in [0, 1]$.

The conformance relations presented in this paper correspond to the classical notions of trace inclusion and trace equivalence. The first one is easier and corresponds to part 1 of Definition 11. Trace inclusion requires that all traces of the implementation are traces of the specification. In our case, we do not have a crisp membership relation. Instead we have the certainty values, a t -norm, and a threshold. Hence, we compose the uncertainty values with the t -norm, and that the trace is correct if the obtained value is above the given threshold.

The relation corresponding to trace equivalence is in part 2 of Definition 11. It cannot be just trace equivalence. Let us suppose that the specification allows the execution of action a in a given location with the fuzzy constraint $\overline{x \leq 3}^{0.2}$ and the threshold is 0.9. In this case, we allow an implementation to perform the action a at time 3.01. But we do not want to force a correct implementation to perform that action at that time. That is, an implementation that always performs action a in the interval $[0, 3]$ cannot be considered incorrect. To avoid this problem we require trace equivalence for the kernel (Definition 1) of the constraints and just trace inclusion for the rest of the elements.

Definition 11 Let A be a timed automaton, fA be a fuzzy-timed automaton, Δ_1 and Δ_2 be t -norms, and $\alpha \in [0, 1]$.

1. A is Δ_1 -conformante with respect to the Δ_2 -operational semantics of fA with an alpha cut of α , denoted by $A \text{ fconf}_{\Delta_1, \Delta_2}^\alpha fA$, if for all trace $(d_1, a_1) \dots (d_n, a_n) \in \text{tr}(A)$ there exist a trace $(d_1, \alpha_1, a_1, \beta_1) \dots (d_n, \alpha_n, a_n, \beta_n) \in \text{ftr}_{\Delta_2}(fA)$ such that $\llbracket \Delta_1 \rrbracket(\alpha_1, \beta_1, \alpha_2, \beta_2, \dots, \alpha_n, \beta_n) \geq \alpha$.
2. A is maximally Δ_1 -conformante with respect to the Δ_2 -operational semantics of fA with an alpha cut of α , written $A \text{ fconfm}_{\Delta_1, \Delta_2}^\alpha fA$, if $A \text{ fconf}_{\Delta_1, \Delta_2}^\alpha fA$ and for any fuzzy trace $(d_1, 1, a_1, 1) \dots (d_n, 1, a_n, 1) \in \text{ftr}_{\Delta_2}(fA)$ we have $(d_1, a_1) \dots (d_n, a_n) \in \text{tr}(A)$.

□

Our upgrade algorithm (presented in Figure 2) plays two different roles: it updates the definition of the behaviors and the fuzzy-time information of the specification.

This algorithm receives four parameters. The first one is the model that is going to be updated. The second one is the trace that will be used to upgrade the model. The last pairs of input parameters are t -norms used to update the fuzzy values in order to provide the conformance relations presented in Definition 11. The algorithm traverses the trace $(d_1, a_1)(d_2, a_2) \dots (d_n, a_n)$ and checks if there exists a trace of the automata that correspond to this trace. Let us note that a list of possible states is needed due to the fact that the specification may be non-deterministic. If there exists a minimal length subtrace $(d_i, a_i) \dots (d_j, a_j)$ with $1 \leq i < j \leq n$, that does not match any trace of the specification, then this trace $(d_j, a_j) \dots (d_n, a_n)$, is added by using new states from the current list of states.

If a pair (d_i, a_i) does not represent a new behavior, then the fuzzy value of both the initial state and the guard of the transition are upgraded by using the third and forth parameters of the algorithm, respectively. Let us note again that we need two t -norms due to Definition 11. Moreover, if the token (d_i, a_i) is new, then a new fuzzy-function is generated by using the second t -norm, and the information of the initial state is updated by using the first t -norm.

The complexity of this algorithm is in the worst case if the specification is deterministic in $O(n)$, where n is the number of tokens of the trace. The cost for non-deterministic specifications is in $O(n \cdot |L| \cdot |E|)$, where $|L|$ is the number of states of the specification and $|E|$ is the number of transitions.

5 Conclusions and Future Work

In this paper we have presented a novel approach to self-adapt a fuzzy-time formal model. During software development, to have an up-to-date formal model specification is hard. In real situations, the implementation is changing but the specification does not reflect these new features. A low intrusive methodology to update the specification of the system is presented. This upgrade process has been implemented by using an algorithm that updates the requirements, both behavioral and temporal, of the specification. The algorithm needs traces generated by the implementation and processes the old model in order to conform the implementation.

As future work, we would like to compare this approach with others, in order to evaluate the quality of

```

def upgrade(fA, t, tnorm1, tnorm2):
    @type fA: fuzzy-automaton (the specification)
    @type t: collected trace
    @type tnorm1: t-norm
    @type tnorm2: t-norm
    @rtype: This function returns a fuzzy-automaton
    list_states = [fA.initial_state]
    # the initial state is introduced
    for i in t:
        # i = (d_i, a_i) is a time value
        # and an action value.
        list_new_states = []
        for s in list_states:
            tr = getTransitions(fA, s, a_i)
            # tr is a list of transitions of fA
            # such that the initial state is s
            # and the action is a_i
            for trans in tr:
                list_new_states.append(trans.goal_state)
                trans.updateFuzzy(d_i, tnorm1, tnorm2)
            #there are updated both:
            #the invariant of the initial state
            #and the trigger condition
        #if tr is the empty set => list_new_states=[]
        if list_new_states == []:
            #then new paths are generated
            #from any state in list_states
            for s in list_states:
                s1 = new_state(fA)
                FF = fuzzy(d_i, tnorm2)
                #generates a fuzzy function with d_i value
                #to trigger the transition
                updateFuzzy(s, d_i, tnorm1)
                #update the invariant location
                #of the initial state
                fA.createTr(s, i.action, s1, FF)
                #is introduced into fA the
                #transition with initial state s,
                #action a_i, goal state s1, and
                #FF as time information.
                list_new_states.append(s1)
            list_states = list_new_states
        #the set of states is updated
    return fA

```

Figure 2. Upgrade algorithm.

the obtained model. Among the alternatives to implement our approach we can mention: ant colony optimization, which uses ants to traverse the solution space and find productive areas. Another approach consists in adding entropy with respect to a model in order to use a cross-entropy method to generate candidate solutions via a parameterized probability distribution.

Acknowledgements

We would like to thank the anonymous reviewers of this paper for their valuable comments.

References

- [1] R. Alur and D. Dill. A theory of timed automata. *Theoretical Computer Science*, 126:183–235, 1994.
- [2] D. Clarke and I. Lee. Automatic generation of tests for timing constraints from requirements. In *3rd Workshop on Object-Oriented Real-Time Dependable Systems, WORDS'97*, pages 199–206. IEEE Computer Society Press, 1997.
- [3] J. Davies and S. Schneider. A brief history of timed CSP. *Theoretical Computer Science*, 138:243–271, 1995.
- [4] A. En-Nouaary, R. Dssouli, and F. Khendek. Timed Wp-method: Testing real time systems. *IEEE Transactions on Software Engineering*, 28(11):1024–1039, 2002.
- [5] A. Hessel, K.G. Larsen, M. Mikucionis, B. Nielsen, P. Pettersson, and A. Skou. Testing real-time systems using UPPAAL. In *Formal Methods and Testing, LNCS 4949*, pages 77–117. Springer, 2008.
- [6] R.M. Hierons, K. Bogdanov, J.P. Bowen, R. Cleaveland, J. Derrick, J. Dick, M. Gheorghe, M. Harman, K. Kapoor, P. Krause, G. Luettgen, A.J.H. Simons, S. Vilkomir, M.R. Woodward, and H. Zedan. Using formal methods to support testing. *ACM Computing Surveys*, 41(2), 2009.
- [7] R.M. Hierons, J.P. Bowen, and M. Harman, editors. *Formal Methods and Testing, LNCS 4949*. Springer, 2008.
- [8] R.M. Hierons, M.G. Merayo, and M. Núñez. Testing from a stochastic timed system with a fault model. *Journal of Logic and Algebraic Programming*, 78(2):98–115, 2009.
- [9] P. Julián, G. Moreno, and J. Penabad. On the declarative semantics of multi-adjoint logic programs. In *10th Int. Conf. on Artificial Neural Networks, IWANN'09, LNCS 5517*, pages 253–260. Springer, 2009.
- [10] D. Lee and M. Yannakakis. Principles and methods of testing finite state machines: A survey. *Proceedings of the IEEE*, 84(8):1090–1123, 1996.
- [11] D. Mandrioli, S. Morasca, and A. Morzenti. Generating test cases for real time systems from logic specifications. *ACM Transactions on Computer Systems*, 13(4):356–398, 1995.
- [12] J. Medina, M. Ojeda-Aciego, and P. Vojtás. Multi-adjoint logic programming with continuous semantics. In *6th Int. Conf. on Logic Programming and Nonmonotonic Reasoning, LPNMR '01, LNCS 2173*, pages 351–364. Springer, 2001.
- [13] M.G. Merayo, M. Núñez, and I. Rodríguez. Extending EFSMs to specify and test timed systems with action durations and timeouts. *IEEE Transactions on Computers*, 57(6):835–848, 2008.
- [14] M.G. Merayo, M. Núñez, and I. Rodríguez. Formal testing from timed finite state machines. *Computer Networks*, 52(2):432–460, 2008.
- [15] S. Munoz-Hernandez, V.P. Ceruelo, and H. Strass. RFuzzy: An expressive simple fuzzy compiler. In *10th Int. Conf. on Artificial Neural Networks, IWANN'09, LNCS 5517*, pages 270–277. Springer, 2009.
- [16] X. Nicollin and J. Sifakis. The algebra of timed processes, ATP: Theory and application. *Information and Computation*, 114(1):131–178, 1994.
- [17] Y. Ortega-Mallén. Operational semantics for timed observations. In *2nd Int. Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems, FTRTFT'91, LNCS 571*, pages 507–527. Springer, 1991.
- [18] I. Rodríguez. A general testability theory. In *20th Int. Conf. on Concurrency Theory, CONCUR'09, LNCS 5710*, pages 572–586. Springer, 2009.
- [19] J. Springintveld, F. Vaandrager, and P.R. D'Argenio. Testing timed automata. *Theoretical Computer Science*, 254(1-2):225–257, 2001. Previously appeared as Technical Report CTIT-97-17, University of Twente, 1997.
- [20] M. Utting and B. Legeard. *Practical Model-Based Testing: A Tools Approach*. Morgan-Kaufmann, 2007.
- [21] Y. Wang, M.Ü. Uyar, S.S. Batth, and M.A. Fecko. Fault masking by multiple timing faults in timed EFSM models. *Computer Networks*, 53(5):596–612, 2009.
- [22] W. Yi. CCS+ Time = an interleaving model for real time systems. In *18th Int. Colloquium on Automata, Languages and Programming, ICALP'91, LNCS 510*, pages 217–228. Springer, 1991.
- [23] L.A. Zadeh. Is there a need for fuzzy logic? *Information Sciences*, 178(13):2751–2779, 2008.